

Hub-Spoke Topology: Enterprise Network Architecture

The scalable pattern enterprises use to centralize shared services and connect dozens of VNets

 Intermediate

 20 Minutes

 AZ-104 · Module 04

Why This Matters

Imagine managing 50 separate office buildings, each with its own security guard, firewall, and reception desk. Nightmare, right? Hub-spoke is how Azure organizations scale. One central hub handles shared services (firewall, VPN, DNS) and spokes connect to it like branches to a tree. Every major enterprise uses this pattern. Understanding it unlocks most exam scenarios.

Before You Start

Prerequisites	VNets & Subnets, Network Security Groups, Routing Fundamentals, VNet Peering, VPN & ExpressRoute
Time to understand	20 minutes
Difficulty	 Intermediate (combines multiple networking concepts)
What you'll learn	Hub-spoke architecture, when to use it, how to implement at scale

The Simple Idea

What Is Hub-Spoke?

Hub-Spoke = A network architecture where one central VNet (hub) holds shared services, and multiple VNets (spokes) peer to it — spokes don't talk directly to each other, they route

through the hub.

What Goes in the Hub?

Hub VNet contains shared infrastructure:

- **VPN Gateway** (on-premises connection)
- **ExpressRoute Gateway** (dedicated circuit)
- **Azure Firewall** (centralized inspection)
- **DNS Resolver** (shared DNS)
- **Network NVA** (network appliance)

Airline Network (Hub-Spoke):

Old Model (Messy):

NYC ↔ LA ↔ Chicago ↔ Denver ↔ Boston

- └ Every city connected to every other city
- └ Too many routes, hard to manage, expensive X

Hub-Spoke Model (Clean):

```
graph LR
    NYC[NYC (Spoke)] --- Dallas[Dallas (Hub)]
    LA[LA (Spoke)] --- Dallas
    Chicago[Chicago (Spoke)] --- Dallas
    Boston[Boston (Spoke)] --- Dallas
```

All flights go through Dallas hub

- └ Easier to manage ✓
- └ Centralized security/customs ✓
- └ Economies of scale ✓
- └ Spokes don't need direct routes ✓

How Hub-Spoke Works

Architecture Diagram

On-Premises (192.168.0.0/16)



Hub VNet (10.0.0.0/16)

- └─ GatewaySubnet (10.0.1.0/24) – VPN/ExpressRoute Gateway
- └─ AzureFirewallSubnet (10.0.2.0/24) – Firewall (centralized inspection)
- └─ SharedServicesSubnet (10.0.3.0/24) – DNS, shared resources
- └─ ManagementSubnet (10.0.4.0/24) – Admin tools

↗ Peering ↖

/

\

Spoke-Prod

(10.1.0.0/16)

└─ App tier

└─ DB tier

Spoke-Dev

(10.2.0.0/16)

└─ App tier

└─ DB tier

Spoke-Test

(10.3.0.0/16)

└─ App tier

└─ DB tier

Peering Features:

- └─ Hub → Spoke: "Allow gateway transit" ✓
- └─ Spoke → Hub: "Use remote gateway" ✓
- └─ Result: Spokes use hub's VPN to reach on-premises ✓

Step 1: Create Hub VNet

Hub VNet: 10.0.0.0/16

- └─ GatewaySubnet: 10.0.1.0/24
 - └─ Used by: VPN Gateway
- └─ AzureFirewallSubnet: 10.0.2.0/24
 - └─ Used by: Azure Firewall
- └─ SharedServicesSubnet: 10.0.3.0/24
 - └─ Used by: DNS, shared apps
- └─ ManagementSubnet: 10.0.4.0/24
 - └─ Used by: Admin VMs

Step 2: Create Spoke VNets

Spoke-Prod: 10.1.0.0/16

- └─ WebSubnet: 10.1.1.0/24
- └─ AppSubnet: 10.1.2.0/24
- └─ DBSubnet: 10.1.3.0/24

Spoke-Dev: 10.2.0.0/16

└─ WebSubnet: 10.2.1.0/24

└─ AppSubnet: 10.2.2.0/24

└─ DBSubnet: 10.2.3.0/24

Spoke-Test: 10.3.0.0/16

└─ WebSubnet: 10.3.1.0/24

└─ AppSubnet: 10.3.2.0/24

└─ DBSubnet: 10.3.3.0/24

Requirement: No overlapping address spaces ✓

Step 3: Deploy Gateways in Hub

VPN Gateway in Hub:

└─ Deployed in GatewaySubnet (10.0.1.0/24)

└─ Connects to on-premises via Site-to-Site VPN

└─ Peering option: "Allow gateway transit" ENABLED

└─ Result: All spokes can reach on-premises through hub

Azure Firewall in Hub:

└─ Deployed in AzureFirewallSubnet (10.0.2.0/24)

└─ Private IP: 10.0.2.4

└─ All internet traffic from spokes routes through it

└─ Result: Centralized inspection, threat detection ✓

Step 4: Create Peering (Hub ↔ Spokes)

Peering 1: Hub ↔ Spoke-Prod

└─ Hub side: "Allow gateway transit" = YES

└─ Spoke side: "Use remote gateway" = YES

└─ Traffic forwarding: ENABLED

└─ Result: Spoke-Prod routes through hub ✓

Peering 2 & 3: Hub ↔ Spoke-Dev, Hub ↔ Spoke-Test

└─ Same pattern, both connected ✓

Note: Spokes are NOT directly peered to each other ✗

Step 5: Configure Routing

In Hub VNet, create User-Defined Routes:

Route Table "HubRoutes":

- |— 0.0.0.0/0 → Azure Firewall (10.0.2.4)
- | └ All internet traffic through firewall
- |— 192.168.0.0/16 → VPN Gateway
- | └ On-premises traffic through gateway
- |— Apply to: All hub subnets

In Each Spoke, create User-Defined Routes:

Route Table "Prod-Routes":

- |— 0.0.0.0/0 → Hub Firewall (10.0.2.4)
- | └ Internet traffic via hub firewall
- |— 192.168.0.0/16 → Hub VPN Gateway
- | └ On-premises via hub gateway
- |— 10.2.0.0/16 → Hub firewall
- | └ Dev traffic via hub (not direct)
- |— 10.3.0.0/16 → Hub firewall
- | └ Test traffic via hub (not direct)
- |— Apply to: All prod subnets (except spokes already peered)

Result:

- |— Prod internal traffic: Direct (10.1.0.0/16)
- |— Prod → Dev traffic: Through hub firewall ✓
- |— Prod → Internet: Through hub firewall ✓
- |— Prod → On-prem: Through hub VPN ✓

Step 6: Apply NSG Rules

Hub Firewall Subnet NSG:

- |— Allow inbound from all spokes (10.1.0.0/16, 10.2.0.0/16, etc.)
- |— Allow inbound from on-premises (192.168.0.0/16)
- |— Allow outbound to internet
- |— Result: Firewall can inspect all traffic ✓

Spoke-Prod App Subnet NSG:

- |— Allow inbound from Prod Web Subnet (10.1.1.0/24)
- |— Allow inbound from Hub Firewall (10.0.2.4/32)
- |— Deny all other inbound
- |— Result: App layer only accessible from web tier and hub ✓

Spoke-Dev App Subnet NSG:

- |— Allow inbound from Dev Web Subnet (10.2.1.0/24)
- |— Allow inbound from Hub Firewall (10.0.2.4/32)
- |— Deny Dev → Prod (traffic filtered at firewall)
- └ Result: Dev isolated from prod ✓

Mental Model: Hub-Spoke as Postal System

Postal Hub-Spoke Network:**Central Post Office (Hub)**

- |— Sorting facility
- |— Border control (customs/inspection)
- |— International mail gateway
- └ Shared infrastructure

Regional Branches (Spokes)

- |— Branch 1 (Production)
- |— Branch 2 (Development)
- |— Branch 3 (Testing)
- └ All mail between branches goes through central office

Mail Flow:

- └ Branch1 → Central Hub → Inspection → Forward to Branch2 ✓

Benefits:

- |— Central inspection point (security)
- |— Shared border gateway (cost savings)
- |— Easy to add new branches
- └ Scalable architecture ✓

Worked Example: Real Scenario

The Scenario

TechCorp Enterprise:

- Headquarters: Dallas data center (192.168.0.0/16)
- 3 business units: Production, Development, Testing
- Each needs isolated Azure environment
- All need access to on-premises data
- All traffic to internet must be inspected for security

Architecture Design

Hub VNet (10.0.0.0/16) - Dallas region

- ├─ GatewaySubnet (10.0.1.0/24)
 - └─ VPN Gateway (connects to on-premises)
- ├─ AzureFirewallSubnet (10.0.2.0/24)
 - └─ Azure Firewall (centralized inspection)
- ├─ SharedServicesSubnet (10.0.3.0/24)
 - └─ DNS resolver, monitoring agents
- └─ ManagementSubnet (10.0.4.0/24)
 - └─ Admin VM for troubleshooting

Spoke-Prod (10.1.0.0/16) - Production

- ├─ WebSubnet (10.1.1.0/24) – Load balancer, web VMs
- ├─ AppSubnet (10.1.2.0/24) – App servers
- └─ DBSubnet (10.1.3.0/24) – Production databases

Spoke-Dev (10.2.0.0/16) - Development

- ├─ WebSubnet (10.2.1.0/24)
- ├─ AppSubnet (10.2.2.0/24)
- └─ DBSubnet (10.2.3.0/24)

Spoke-Test (10.3.0.0/16) - Testing

- ├─ WebSubnet (10.3.1.0/24)
- ├─ AppSubnet (10.3.2.0/24)
- └─ DBSubnet (10.3.3.0/24)

Step 1: Setup Hub

1. Create Hub VNet (10.0.0.0/16)
2. Create 4 subnets (Gateway, Firewall, SharedServices, Management)
3. Deploy VPN Gateway in GatewaySubnet
 - └─ Configure for on-premises connection

4. Deploy Azure Firewall in AzureFirewallSubnet
 - └─ Assign static public IP
5. Create DNS Resolver in SharedServicesSubnet
 - └─ For hybrid DNS resolution

Step 2: Setup Spokes

1. Create 3 spoke VNets:
 - └─ Prod (10.1.0.0/16)
 - └─ Dev (10.2.0.0/16)
 - └─ Test (10.3.0.0/16)
2. In each spoke, create 3 subnets:
 - └─ Web (Web servers / load balancer)
 - └─ App (Application servers)
 - └─ DB (Databases)
3. Verify: No overlapping address spaces ✓

Step 3: Create Peerings

Peering: Hub ↔ Prod

- └─ Hub setting: "Allow gateway transit" = YES
- └─ Prod setting: "Use remote gateway" = YES
- └─ Traffic forwarding: ENABLED
- └─ Status: Connected ✓

Peering: Hub ↔ Dev

- └─ Hub setting: "Allow gateway transit" = YES
- └─ Dev setting: "Use remote gateway" = YES
- └─ Traffic forwarding: ENABLED
- └─ Status: Connected ✓

Peering: Hub ↔ Test

- └─ Hub setting: "Allow gateway transit" = YES
- └─ Test setting: "Use remote gateway" = YES
- └─ Traffic forwarding: ENABLED
- └─ Status: Connected ✓

NOT Created: Prod ↔ Dev, Prod ↔ Test, Dev ↔ Test

└ Keeps environments isolated ✓

Step 4: Configure Routing in Hub

Destination	Next Hop	Purpose
0.0.0.0/0	Firewall (10.0.2.4)	Internet traffic
192.168.0.0/16	VPN Gateway	On-premises

Apply to all hub subnets.

Step 5: Configure Routing in Spokes

Destination	Next Hop	Purpose
10.0.0.0/16	Hub (peering)	Hub access (peering, not UDR needed)
10.2.0.0/16	Firewall (10.0.2.4)	Dev traffic via firewall
10.3.0.0/16	Firewall (10.0.2.4)	Test traffic via firewall
192.168.0.0/16	VPN Gateway (via hub)	On-premises via hub
0.0.0.0/0	Firewall (10.0.2.4)	Internet via firewall

Apply to all prod subnets. Dev and Test get similar routes (swap Prod with Dev/Test as destination).

Step 6: Configure NSGs

Prod WebSubnet NSG

└ Allow inbound on 80, 443

└ from 0.0.0.0/0 (internet)

└ Allow inbound on port 8080

└ from App subnet (10.1.2.0/24)

└ Deny all other inbound

Prod AppSubnet NSG

```
|— Allow inbound from Web subnet  
|   (10.1.1.0/24)  
|— Allow inbound from Hub Firewall  
|   (10.0.2.4/32)  
|— Allow outbound to DB subnet  
|   (10.1.3.0/24)  
|— Allow outbound to Firewall  
|   (10.0.2.4/32)  
|— Deny all other inbound
```

Prod DBSubnet NSG

```
|— Allow inbound port 1433  
|   from App subnet (10.1.2.0/24)  
|— Deny all inbound from other  
|   spokes (firewall inspects,  
|   can't bypass)  
|— Deny internet access
```

(Dev and Test get similar NSGs)

Hub Firewall Subnet NSG

```
|— Allow inbound from all spokes (10.1.0.0/16, 10.2.0.0/16, 10.3.0.0/16)  
|— Allow inbound from on-prem (192.168.0.0/16)  
|— Allow outbound to internet  
|— Allow outbound to all subnets
```

Step 7: Test Traffic Flows

Scenario 1: Prod VM to Internet

```
|— ProdVM (10.1.2.10) → Destination 8.8.8.8  
|— Routing: 0.0.0.0/0 → Firewall (10.0.2.4)  
|— Path: Prod App → Hub Firewall → Internet
```

- └ Firewall: Inspect traffic ✓
- └ Result: Allowed (if rule permits) ✓

Scenario 2: Prod VM to On-Premises

- └ ProdVM (10.1.2.10) → Server (192.168.1.50)
- └ Routing: 192.168.0.0/16 → VPN Gateway (via hub peering)
- └ Path: Prod App → Hub VPN Gateway → VPN Tunnel → On-prem
- └ Gateway: Encrypt/decrypt ✓
- └ Result: Connected ✓

Scenario 3: Prod VM to Dev VM

- └ ProdVM (10.1.2.10) → DevVM (10.2.2.10)
- └ Routing: 10.2.0.0/16 → Firewall (10.0.2.4)
- └ Path: Prod App → Hub Firewall → Dev App
- └ Firewall: Can block cross-environment traffic ✓
- └ NSG (Dev App): Allows or blocks from firewall
- └ Result: Controlled by firewall rules ✓

Scenario 4: Admin Troubleshooting

- └ Admin in Hub Management subnet (10.0.4.10)
- └ Needs to check Prod DB (10.1.3.20)
- └ Routing: 10.1.0.0/16 → Peered (direct)
- └ NSG: Prod DB may or may not allow management IPs
- └ Result: Depends on NSG policy ✓

Common Mistakes (What NOT to Do)

✗ Mistake 1: Peering All Spokes to Each Other

Wrong

Create peerings:

- | Hub ↔ Prod
- | Hub ↔ Dev
- | Prod ↔ Dev (DIRECT PEERING!)
- | Prod ↔ Test
- | Dev ↔ Test
- └ Too many connections to manage X

Problems:

- | Lost central inspection (Prod→Dev bypasses firewall)
- | Hard to scale (add 4th spoke = 3 new peerings)
- | Security risk (no firewall filter)
- └ Maintenance nightmare X

Fix

Only peer spokes to hub:

- | Hub ↔ Prod ✓
- | Hub ↔ Dev ✓
- | Hub ↔ Test ✓
- └ Spokes don't talk directly

All spoke-to-spoke traffic routes
through firewall ✓

Why it fails: Defeats the purpose of hub-and-spoke architecture.

✗ Mistake 2: Forgetting Gateway Transit Settings

Wrong

Peering created: Hub ↔ Prod

- | "Allow gateway transit": NOT enabled on hub
- | "Use remote gateway": Enabled on prod
- | Prod VM tries to reach on-premises
- | Traffic routes to hub, but hub can't
| forward to VPN
- | Result: Connection fails X

Fix

Peering: Hub ↔ Prod

- └ Hub side: "Allow gateway transit" = YES
- └ Prod side: "Use remote gateway" = YES
- └ Both must be set correctly
- └ Now prod can use hub's VPN ✓

Why it fails: Both sides of the peering need compatible settings.

✗ Mistake 3: Non-Transitive Routing

Wrong

Assumption: "If Hub ↔ Prod and Hub ↔ Dev are peered, Prod and Dev can talk"

Reality:

- └ Peering is non-transitive
- └ A ↔ B and B ↔ C doesn't mean
- └ A ↔ C automatically ✗
- └ Prod can reach Hub
- └ Dev can reach Hub
- └ But Prod→Dev traffic doesn't
- └ automatically work
- └ Need explicit UDR routing through firewall ✓

Fix

Configure UDRs in spokes:

- └ Prod Route Table: 10.2.0.0/16 → Firewall
- └ Dev Route Table: 10.1.0.0/16 → Firewall
- └ Now traffic goes: Prod → Firewall → Dev
- └ Firewall can inspect and allow/block
- └ Transitive connectivity achieved ✓

Why it fails: Peering doesn't transitively connect spokes.

✗ Mistake 4: Overlapping Address Spaces

Wrong

```
Hub: 10.0.0.0/16
Prod: 10.0.0.0/16 (SAME!)
Dev: 10.2.0.0/16

Try to peer:
└─ Hub ↔ Prod: FAILS (overlapping) ✗
└─ Hub ↔ Dev: OK
└─ Cannot achieve full hub-spoke ✗
```

Fix

```
Plan CIDR ranges upfront:
└─ Hub: 10.0.0.0/16
└─ Prod: 10.1.0.0/16 ✓
└─ Dev: 10.2.0.0/16 ✓
└─ Test: 10.3.0.0/16 ✓
└─ On-prem: 192.168.0.0/16
└─ All peering works ✓
```

Why it fails: Overlapping addresses cause routing ambiguity.

Hub-Spoke Checklist

Planning

- Identify hub location (central region, on-premises connection point)
- Plan CIDR ranges (hub + all spokes, no overlaps)
- Decide on shared services (VPN, firewall, DNS)
- Document traffic flow policies (who talks to whom)

Hub Setup

- ❑ Create Hub VNet with planned CIDR
- ❑ Create GatewaySubnet (for VPN/ExpressRoute)
- ❑ Create AzureFirewallSubnet (if using firewall)
- ❑ Create SharedServicesSubnet (DNS, shared resources)
- ❑ Deploy VPN/ExpressRoute Gateway (if hybrid needed)
- ❑ Deploy Azure Firewall (if centralized inspection needed)
- ❑ Create Hub Route Table (internet → firewall, on-prem → gateway)

Spoke Setup

- ❑ Create Spoke VNets (non-overlapping CIDR)
- ❑ Create subnets (Web, App, DB or similar tiers)
- ❑ Create Spoke Route Tables (internet/other-spokes/ on-prem → firewall)

Peering

- ❑ Peer Hub ↔ Spoke-1
 - └ Hub: "Allow gateway transit" = YES
 - └ Spoke-1: "Use remote gateway" = YES
- ❑ Peer Hub ↔ Spoke-2
 - └ Hub: "Allow gateway transit" = YES
 - └ Spoke-2: "Use remote gateway" = YES
- ❑ Peer Hub ↔ Spoke-3 (repeat pattern)
- ❑ Verify: Peering status = Connected

NSG Rules & Testing

- ❑ Hub: Allow traffic from all spokes and on-prem
- ❑ Spokes: Allow traffic from hub firewall only
- ❑ Spokes: Allow internal subnet traffic (Web→App→DB)
- ❑ Spokes: Block direct spoke-to-spoke (force through hub)

- ❑ Test spoke → hub connectivity (peering works)
- ❑ Test spoke → internet (routes through firewall)
- ❑ Test spoke → on-prem (uses hub gateway)

- Test spoke → other-spoke (routes through firewall)
- Test firewall rules (inspect and allow/block)

How This Connects to Other Topics

Related to Module 01 (Identity & Governance)

- **RBAC:** Only network admins can create peerings, modify hub routes
- **Policy:** Enforce hub-and-spoke topology for all VNets

Related to Module 02 (Storage)

- **Storage Access:** Spokes access storage through hub firewall

Related to Module 03 (Compute)

- **VM Communication:** VMs in different spokes communicate via hub

Related to Module 05 (Monitor)

- **Monitor Hub:** Track traffic through firewall and gateways
- **Network Watcher:** Diagnose spoke-to-spoke connectivity issues

See It In Action

Associated labs:

- Lab 12: VNets & Subnets
- Lab 13: NSGs
- Lab 14: VNet Peering
- Lab 15: Azure Firewall

Suggested learning sequence:

1. ☒ Read **VNets & Subnets**
2. ☒ Read **NSGs**
3. ☒ Read **Routing Fundamentals**

4.  Read VNet Peering
5.  Read VPN & ExpressRoute
6.  Read this doc (Hub-Spoke Topology)
7.  Work through Labs 12-15 (implement hub-spoke)

Key Takeaways

Summary

- **Hub-Spoke = scalable network architecture** (one hub, many spokes)
- **Hub contains shared services** (VPN, firewall, DNS, shared resources)
- **Spokes don't peer directly** (all traffic through hub for inspection)
- **Gateway transit** = spokes use hub's VPN/ExpressRoute
- **Routing is key** = UDRs direct spoke-to-spoke through firewall
- **Non-transitive peering** = $A \leftrightarrow B$ and $B \leftrightarrow C$ doesn't mean $A \leftrightarrow C$
- **Plan CIDR upfront** = No overlapping address spaces
- **NSGs enforce policy** = Control which traffic is allowed

Next Steps

1. **Learn:** Read this doc (you're here)
2. **Design:** Draw your hub-spoke topology (CIDR, subnets, peerings)
3. **Implement:** Work through Labs 12-15 (hands-on setup)
4. **Monitor:** Track traffic flows through hub firewall
5. **Scale:** Add spokes as new business units need Azure environments

© 2026 — AZ-104 Learning Hub. For study purposes only.